

STUDENT PERFORMANCE MANAGEMENT SYSTEM DOCUMENTATION

Semester GPA Calculator

A lightweight command-line application written in Python for calculating semester Grade Point Average (GPA) using the grading scale implemented in this project for **University**.

The application accepts a student's courses, credit units, and final percentage scores, calculates the corresponding grade points, and produces a formatted academic performance summary.

Features

- Calculate semester GPA on a 5.00 scale.
- Convert percentage scores into letter grades and grade points.
- Support multiple courses per semester.
- Validate numeric input and score ranges.
- Validate student names and course codes.
- Calculate credit-weighted GPA.
- Display a formatted academic summary.
- Object-oriented course representation using Python [dataclasses](#).
- Graceful handling of invalid input and [Ctrl+C](#).

Grading Scale

The following grading scale is implemented by the application:

Percentage	Letter Grade	Grade Point
80 – 100	A	5.0
75 – 79.99	B+	4.5
70 – 74.99	B	4.0
65 – 69.99	C+	3.5
60 – 64.99	C	3.0
55 – 59.99	D+	2.5
50 – 54.99	D	2.0
0 – 49.99	F	0.0

Note: This table reflects the grading rules currently implemented in this project. Users should verify the applicable **University** academic regulations before using the application for official academic decisions.

GPA Calculation

The semester GPA is calculated using a credit-weighted average:

$$\text{GPA} = \frac{\sum(\text{Grade Point} \times \text{Credit Units})}{\sum(\text{Credit Units})}$$

Example

Suppose a student has:

Course	Credit Units	Score	Grade	Grade Point
CS1101	3	82	A	5.0
MA1101	3	73	B	4.0
PHY1101	2	67	C+	3.5

The GPA is:

$$(5.0 \times 3 + 4.0 \times 3 + 3.5 \times 2) / (3 + 3 + 2)$$

$$= 36 / 8$$

$$= 4.50$$

Therefore:

$$\text{Semester GPA} = 4.50 / 5.00$$

Requirements

Python 3.10 or newer

No third-party packages are required for the basic CLI version.

Installation

1. Clone the repository

```
git clone https://github.com/okirorinnocent/SEMESTER1_PROJECT1
```

2. Optional: Create a virtual environment

Linux/macOS:

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows:

```
python -m venv .venv
.venv\Scripts\activate
```

3. Run the application

```
python student_performance_management_system.py
```

Usage

Start the application:

```
python student_performance_management_system.py
```

You will be prompted for:

1. Student's full name.
2. Number of courses taken.
3. Course code.
4. Credit units.
5. Final percentage score.

Example:

```
=====
UNIVERSITY SEMESTER ACADEMIC PERFORMANCE CALCULATOR
=====
```

Enter Student Full Name: John Doe

Enter number of courses taken this semester: 3

--- Course 1 Entry ---

Course Code (e.g., CS1101): CS1101

Credit units: 3
Final Score Percentage (0-100): 82

--- Course 2 Entry ---
Course Code (e.g., CS1102): CS1102
Credit units: 3
Final Score Percentage (0-100): 74

--- Course 3 Entry ---
Course Code (e.g., MA1101): MA1101
Credit units: 2
Final Score Percentage (0-100): 68

The application then displays a summary:

=====				
ACADEMIC SUMMARY FOR: JOHN DOE				
=====				
CODE	CREDITS	SCORE	GRADE	GP

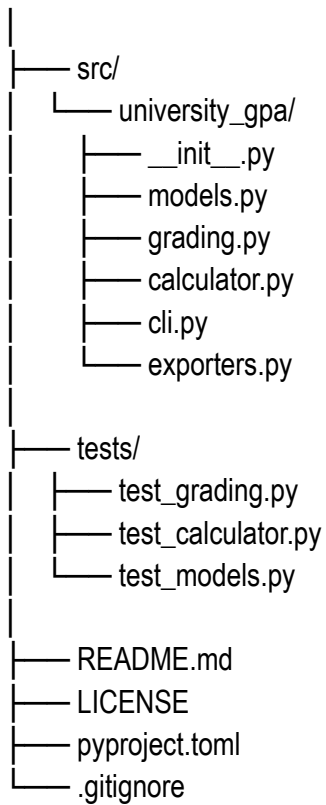
CS1101	3	82.0	A	5.0
CS1102	3	74.0	B	4.0
MA1101	2	68.0	C+	3.5

Total Credit Units : 8				
Semester GPA : 4.19/5.00				
=====				

Project Structure

A recommended production-oriented structure is:

[university-gpa-calculator/](#)



Module responsibilities

models.py

Contains application data models such as:

Course
Semester
Student

grading.py

Contains grading rules:

`calculate_grade_point()`

calculator.py

Contains GPA and CGPA calculations:

`calculate_gpa()`

`calculate_cumulative_gpa()`

cli.py

Handles:

- Terminal input.
- Input validation.
- Terminal output.
- User interaction.

exporters.py

Handles exporting academic results to formats such as:

CSV

JSON

tests/

Contains automated unit and integration tests.

Development

Run the application:

```
python main.py
```

Run tests after installing **pytest**:

```
pytest
```

For improved code quality, consider adding:

```
pytest
```

```
ruff
```

```
black
```

```
mypy
```

These tools can provide automated testing, linting, formatting, and static type checking.

Future Improvements

Planned or possible improvements include:

- CSV result export.
 - JSON result export.
 - Cumulative GPA/CGPA calculation.
 - Student registration number support.
 - Semester and academic-year tracking.
 - SQLite database storage.
 - Automated unit tests.
 - Command-line arguments.
 - Interactive menu system.
 - Web interface.
 - Desktop GUI.
 - PDF academic report generation.
 - GitHub Actions CI/CD.
-

Example Future CLI

A more advanced version could support commands such as:

```
python -m university_gpa calculate
```

or:

```
python -m university_gpa calculate --student "John Doe"
```

and eventually:

```
python -m university_gpa export --format json
```

Contributing

Contributions are welcome.

A typical contribution workflow is:

```
git checkout -b feature/my-feature
```

Make your changes, add tests where appropriate, and submit a pull request.

Please ensure that the code:

- Follows PEP 8.
 - Includes appropriate type hints.
 - Includes tests for new calculation logic.
 - Does not introduce changes to the grading rules without documentation.
-

License

This project is distributed under the MIT License. See [LICENSE](#) for details.

Disclaimer

This software is an educational/project implementation and should not be considered an official **UNIVERSITY** academic records system.

Always verify GPA, grading rules, and academic results against official university records and applicable academic regulations.

Author

Okiror Innocent

Computer Science Student at Mbarara University of Science and Technology (MUST)